

TMT Intelligence Console

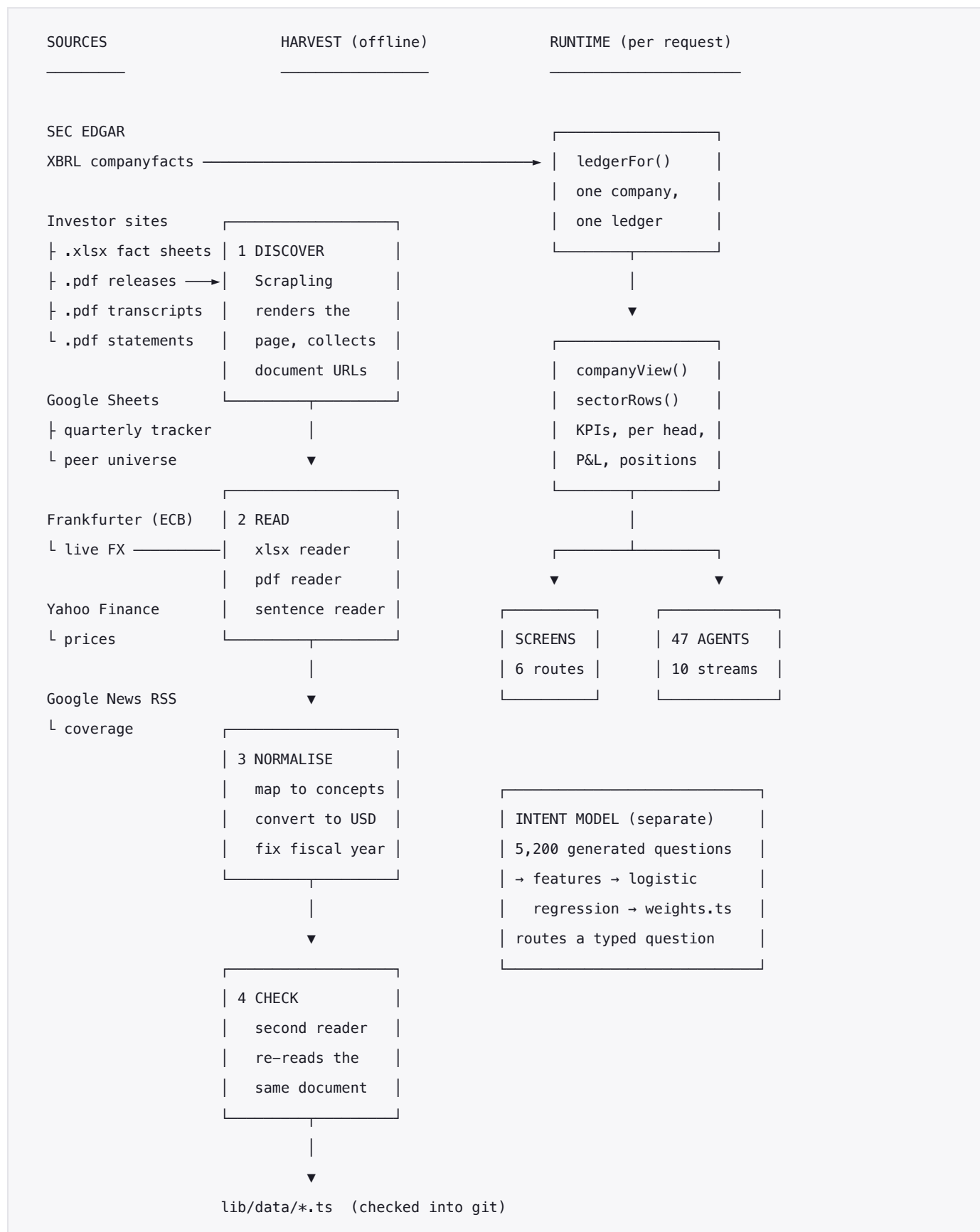
System overview and data flow

A due-diligence console for technology, media and telecom. It reads what companies published, computes every figure itself, and shows the document behind each number.

Seventy companies. Forty nine file with the SEC. The rest, including most of the Indian coverage, publish on their own investor-relations sites and are read from there.

1. No figure is typed in by hand or estimated. Every number traces to a document the system downloaded and parsed.
 2. If a figure is not published, the screen says so. It is never filled in with a guess, an average or a model output.
-

1. The pipeline end to end



The left half runs when someone refreshes the data, roughly once a quarter after results season. The right half runs on every request.


```
npm run discover:ir
```

4. Stage 3: reading the documents

Three readers, tried in order of how structured the source is.

Reader 1: spreadsheets

`lib/research/xlsx.ts` unzips the workbook and walks the sheet XML directly. No dependency.

`parseWorkbookMetrics()` then finds the period header row and reads the grid. Two cases it handles that a naive reader gets wrong:

- **Excel date serials as headers.** `45838` is a date, not a value. Detected

by checking the spacing between consecutive serials is a month, a quarter, a half or a year.

- **Two-row headers.** A data sheet often puts the fiscal year on one row and

the quarter on the next, with the year written once per group of five columns. The two rows are paired back together so each column carries a whole period. This is what unlocked Tech Mahindra: 0 measures before, 18 after, with history back to FY2016.

Reader 2: PDFs

`lib/pdf/extract.ts` is a PDF reader written for this project. It handles:

- AES-256 encrypted files (revision 5 and 6) with an empty user password,

which several fact sheets ship as.

- Subset fonts, where the character codes are arbitrary and the real text is

only recoverable through the ToUnicode CMap.

- Text positioning, so a table laid out with coordinates comes back as rows.

The extracted lines are then run through the same grid parser as the spreadsheets.

Reader 3: sentences

Most mid-caps do not publish a grid at all. The quarter is written out in prose, one figure per bullet:

> "For the full year FY26, the company posted revenue of \$643.7M, growing 3.1% > in reported currency."

`lib/research/ir-narrative.ts` reads those. A figure is taken **only** when a single sentence satisfies all four conditions:

CONDITION	WHY
Names the period	otherwise the figure has no date
Names the measure	otherwise it is an unlabelled number
Carries a currency marker	\$, INR , Rs , EUR
Carries a scale	M , million , bn , crore , lakh

It also repairs a layout artefact: PDF engines strand one or two letters at the end of a line, so "net cash" arrives as "net c" then "ash". A stranded fragment is welded back on; an ordinary line wrap keeps its space.

5. Stage 4: normalising into one ledger

Everything above produces rows with a label, a period and a number. Stage 4 turns those into the same 44-concept ledger the SEC path produces, so the rest of the system never needs to know where a figure came from.

`lib/research/ir-facts.ts` does four things.

maps to revenue and "Total operating expenses" does not map to cost of revenue.

the one its *current* results are stated in, ranked by: does this currency have revenue, is its revenue the most recent, how many measures does it carry, then USD as a tiebreak. This is what stops a 2016 IND-AS adoption file in rupees outvoting a current data sheet in dollars.

date on the provenance line.

Q1 FY27 to June 2026. Without this, every Indian period sat up to nine months in the future.

period label	fiscal year end	calendar end
"FY2026"	March	2026-03-31
"Q1 FY2027"	March	2026-06-30
"FY2026"	December	2026-12-31

not state is dropped rather than assumed. And a value less than a fortieth of another period in the same file is dropped, because a quarter is never that small next to its neighbours: that pattern means a column slipped.

6. Stage 5: the checked reading

Some investor-relations hosts serve their files to an ordinary connection and refuse one originating from a data centre. Verified directly: the same URL with the same headers returns the spreadsheet from a home network and nothing from a hosting provider. No amount of header work changes it.

So the latest published quarter for those companies is read once, offline, and then **read again by a different reader against the same document** before it is recorded.

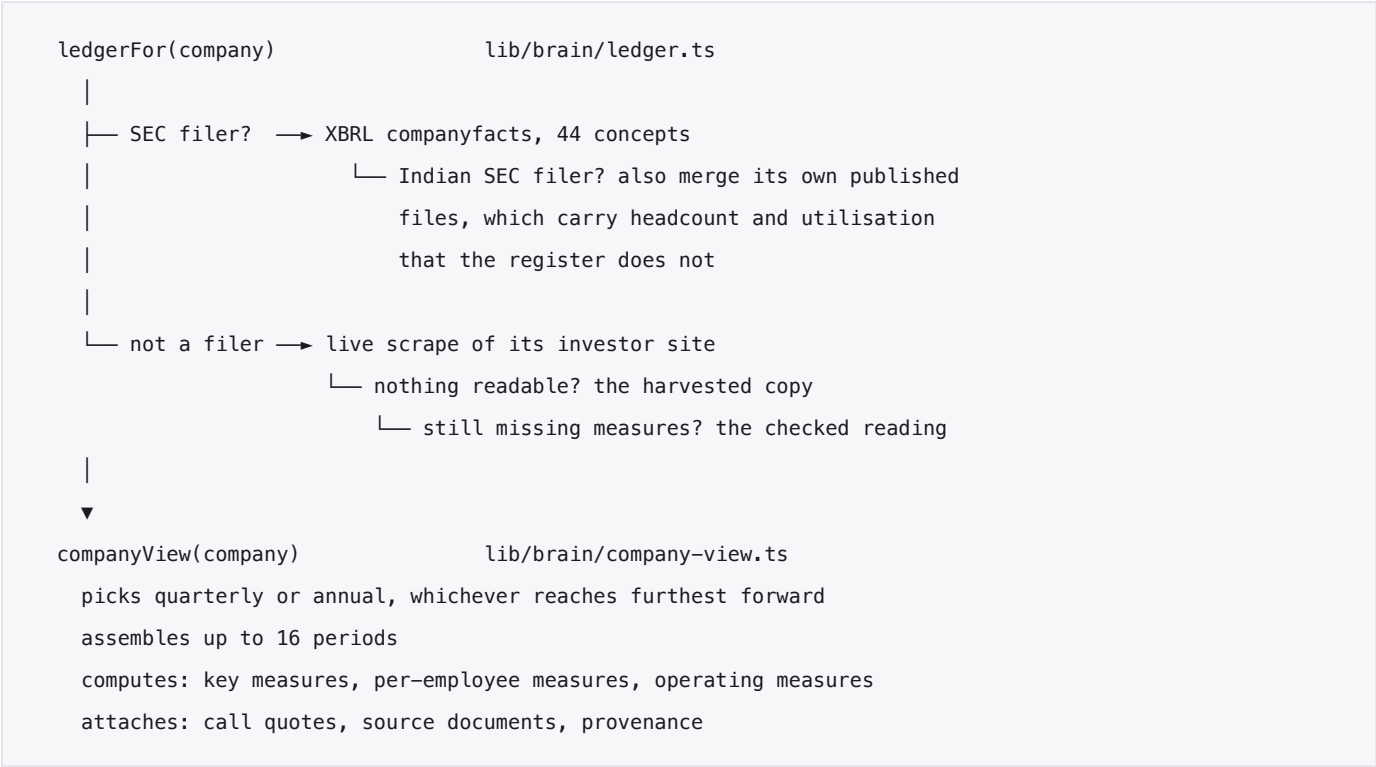
```
reader → record: period, revenue, margin, PAT, headcount, source URL
      |
      ▼
checker → fetches the SAME document independently
        checks: is the period right?
                  is the scale right? (crore to millions is the usual error)
                  consolidated or standalone?
                  is any figure absent from the cited document?
        |
        ├── confirmed → recorded
        ├── corrected → recorded with the correction applied
        └── refuted   → dropped
        |
        ▼
lib/data/ir-published.ts
```

Result: 19 companies, 17 confirmed, 2 corrected on the second reading.

The same check runs on earnings call quotes, but deterministically: `scripts/write-calls.mts` downloads each transcript and searches it for the exact sentence. 27 of 40 quotes were found and kept; 13 were dropped. Two companies set their transcripts in two columns, which the PDF reader interleaves, so nothing of theirs could be confirmed and they carry no quotes.

those documents on every request. The stored reading only ever fills a gap that the live read left, and a live figure always wins.

7. Stage 6: what runs on a request



The per-employee measures

These are the KPIs a services business is actually judged on, and none of them is a filed concept. Each is computed and states its own arithmetic on the card.

MEASURE	COMPUTED AS
Revenue per employee	revenue for the year over closing headcount
Cost of services per employee	cost of services for the year over closing headcount
SG&A per employee	SG&A for the year over closing headcount
Operating profit per employee	operating income for the year over closing headcount
Revenue per billable employee	revenue over headcount multiplied by published utilisation

Utilisation is never estimated. A company that does not publish it carries no revenue-per-billable figure.

Live example:

	PERIOD	REVENUE/EMPLOYEE	REV/BILLABLE	UTILISATION
Infosys	FY2025	\$59,607	\$69,311	86.0%
Coforge	Q1 FY27	\$51,242	\$63,028	81.3%
TCS	Q1 FY2027	\$51,358	not published	not published

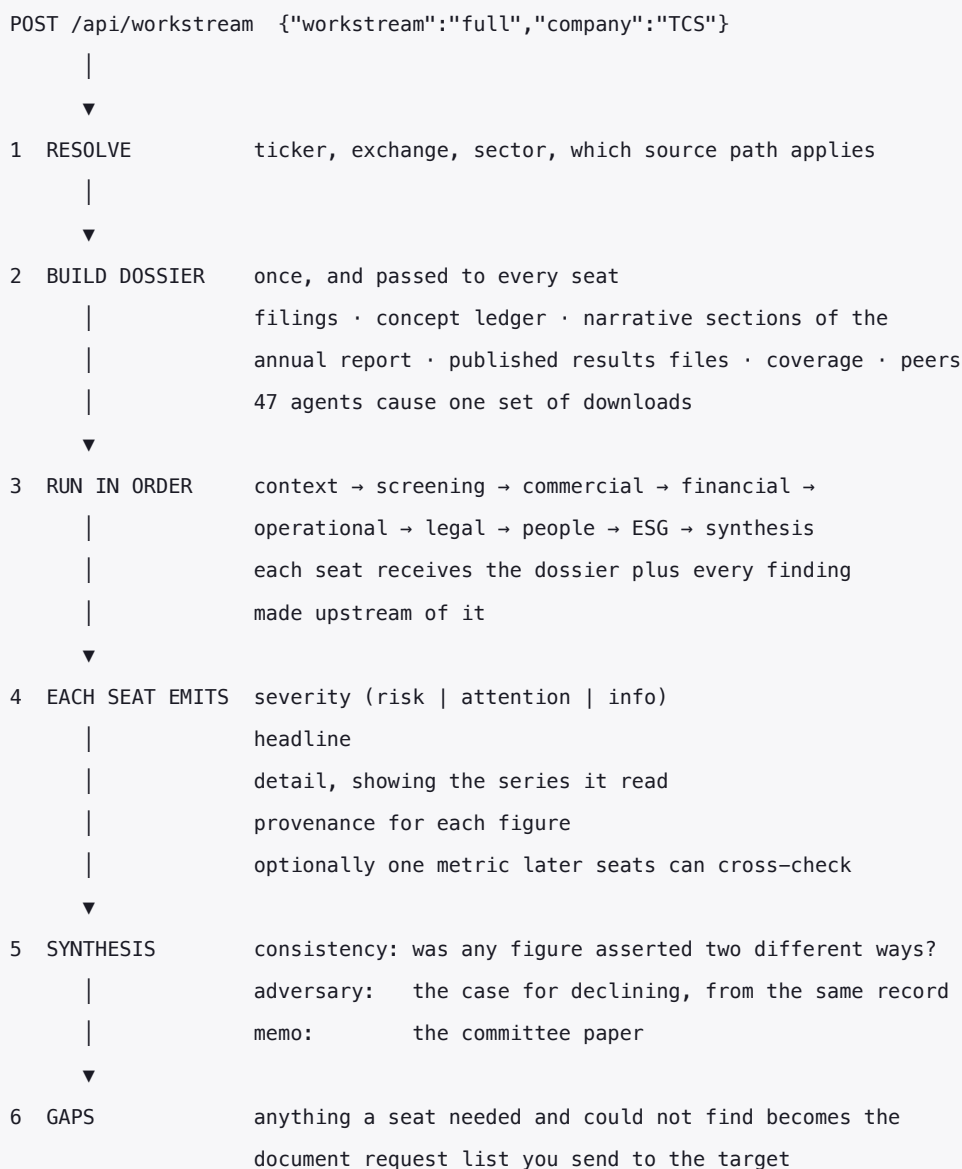
Caching

In-process, six hours, keyed per company and per view. There is no database. Behind a load balancer each instance warms separately, so the first request to each is slow and the rest are not.

8. The agent layer

Forty seven agents across ten workstreams. They are deterministic TypeScript. No model is called, nothing is generated, and the same company on the same filings gives the same answer every time.

They are called agents because each holds a role, reads only the evidence its role calls for, states what it found, and hands to the next.



The rule the whole layer rests on: **a seat may only state something it can point at**. If the evidence is not there it says so, and the gap goes on the list. That is why the console will tell you a figure is not published rather than estimating it.

Three files:

FILE	HOLDS
lib/agents/registry.ts	the 47 seats: id, role, what it needs, who it hands to
lib/agents/analysis.ts	what each seat does with the evidence
lib/agents/os.ts	the orchestrator: order, evidence, gates, handoffs

with the same id to the map at the bottom of `analysis.ts` , and emit with `e.find(severity, headline, detail, metric?)` .

9. The trained model

Separate from everything above. It routes a typed question to the right computation, and it is the only trained component in the system.

run first and produce an intent. The model overrides that intent only when it is confident. Below the confidence floor the rules keep the answer. Measured on held-out phrasings, that combination beats either one alone.

Training flow

```

scripts/train-intent.mts
|
1  GENERATE      116 pattern families, ~200 patterns
|               each filled 26 times from the real universe and
|               the real metric list
|               → 6,084 labelled questions across 6 intents
|
|               "what is {co}'s {metric}"      → metric
|               "{co} versus {co2} on {metric}" → compare
|               "who leads on {metric}"        → rank
|               "{metric} at {co} over {years}" → trend
|
▼
2  SPLIT         held out by FAMILY, not by row
|               34 families, 1,794 examples
|               so the test set contains phrasings the model has
|               never seen, not rewordings of what it trained on
|
▼
3  FEATURES     word unigrams and bigrams
|               character n-grams (3 to 5, with boundaries)
|               first word, first two words
|               structural counts: how many companies, how many metrics
|               2,561 features after pruning at 2 occurrences
|
▼
4  TRAIN        multinomial logistic regression, written out longhand
|               60 epochs · learning rate 2.0 · L2 2e-5 · 6 classes
|
▼
5  EVALUATE     model alone      81.2%
|               rules alone      82.1%
|               model + rules    82.7% at a 0.80 confidence floor
|               the floor is swept from 0 to 0.95 and the best is chosen
|
▼
6  EMIT         lib/brain/intent-model.ts
|               weights as a TypeScript array, committed to git

```

Where the model actually earns its place

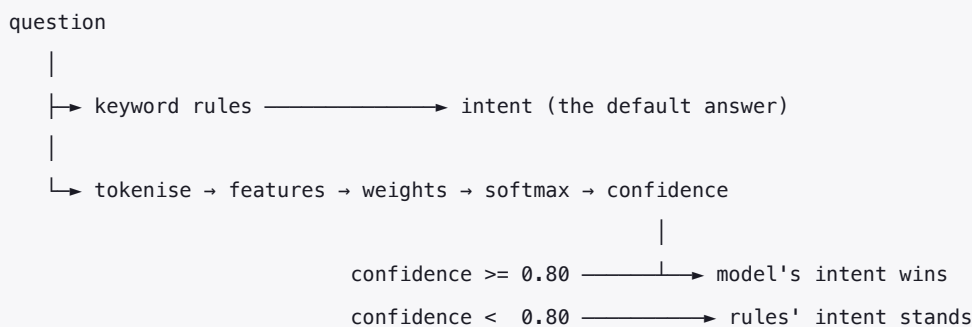
Per class, on held-out families:

INTENT	MODEL	RULES	N
metric	72.7%	90.2%	286
compare	100.0%	74.5%	208
rank	100.0%	99.0%	208
trend	75.0%	80.8%	416
explain	68.8%	68.3%	416
product	90.0%	90.0%	260

The model is decisively better on `compare`, where the rules mishandle phrasings like "how does X stack up against Y". The rules are better on `metric`, which is the plainest case and the one they were written for. Running the model in front of the rules with a confidence floor keeps the win on `compare` without losing the win on `metric`.

hidden the fact that the rules beat the model overall, and that the value is in one class rather than across the board.

At runtime



There is **no model file to load, no inference server and no GPU**. The weights are a TypeScript array and the classifier is about forty lines of arithmetic.

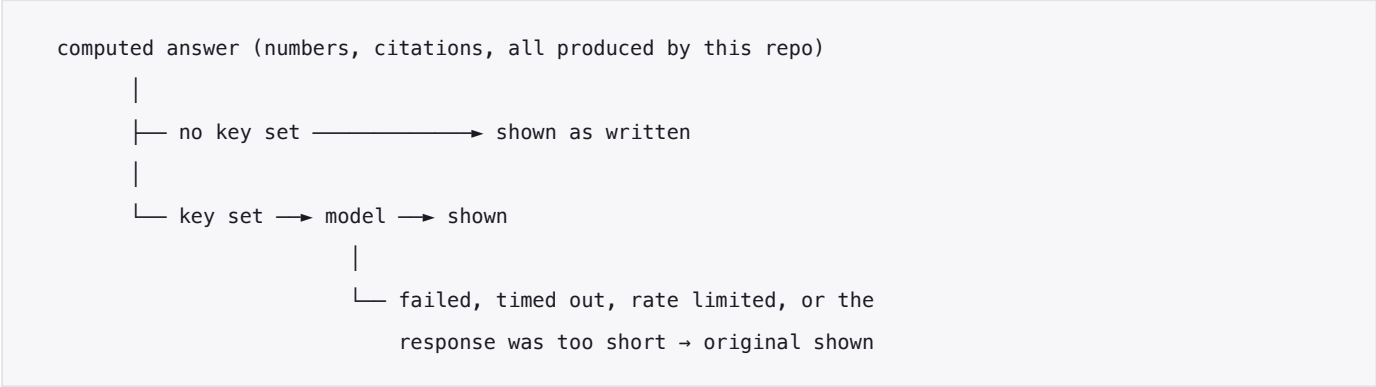
```
npm run train:intent # rewrites lib/brain/intent-model.ts, commit the result
```

FILE	ROLE
<code>scripts/train-intent.mts</code>	generates data, trains, evaluates, sweeps the floor, writes weights
<code>lib/brain/intent-model.ts</code>	the weights, checked in
<code>lib/brain/classifier.ts</code>	feature extraction and inference
<code>lib/brain/intent.ts</code>	the rules, and the floor that decides which answer stands

10. The optional language model

The console needs no key and never will for any figure.

When `GEMINI_API_KEY` or `HF_API_TOKEN` is set, the answer text on the "Ask the data" panel is passed through the model with an instruction to tighten the prose and change no number, name, date or citation. With no key the console writes that text itself.



A dead key cannot empty a page. One file: `lib/ai/provider.ts`.

11. Provenance

Every figure that reaches a screen carries a provenance record. This is enforced by the type system, not by convention.

KIND	MEANS
live	fetches on this request
cached	fetches recently, within the cache window
filing	computed from a filed or published document
baseline	from a stored reading, with the date it was taken
unavailable	could not be obtained, with the reason stated

A `baseline` figure is never shown as though it arrived this morning. The chip on the panel carries the kind and the timestamp.

12. Setting it up on a server, step by step

Two things can be set up here, and they are independent.

needed when you want a newer quarter in the stored floor, roughly once a quarter after results season.

You can run the console for months without ever doing part B.

Part A: run the console

```
git clone https://github.com/soj5-sudo/EY_TMT_Agents.git
cd EY_TMT_Agents
```

```
npm ci
```

Takes about a minute. It installs three packages and their tree. If you see warnings about peer dependencies, ignore them.

```
npm run build
npm run start
```

Open `http://your-server:4870` . That is it. No key, no database, no config file.

filings. It caches for six hours after that.

Part B: refresh the data and retrain the model

Everything below writes files into `lib/data/` and `lib/brain/` . Those files are committed to git. **Deploying the commit is how the new data reaches production.** Nothing is written at runtime.

B1. Set up Python, once

The scraper needs Python 3.10 or newer. Check first:

```
python3 --version
```

If that prints 3.9 or older, install a newer Python before continuing. On macOS the system Python is 3.9 and will not work.

Now create the environment. This lives inside the repo at `scraper/.venv` and is gitignored.

```
cd scraper
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -r requirements.txt
cd ..
```

That installs Scraping, which is the open-source scraper this uses.

in a real browser for the sites that need it, and that browser has to be downloaded once.

```
scraper/.venv/bin/python -m playwright install chromium
```

About 150 MB. On Linux you may also need the system libraries:

```
scraper/.venv/bin/python -m playwright install-deps chromium
```

```
scraper/.venv/bin/python -c "from scrapling.fetchers import DynamicFetcher; print('scraper ready')"
```

You should see `scraper ready`. If you see an import error, the venv did not install; run the pip step again and read its output.

B2. Find the documents

```
npm run discover:ir
```

request first; if that page builds its file list in JavaScript, it renders the page in the browser you just installed and collects the links from the rendered page. Scores every link and keeps the best 24 per company.

TCS.NS	plain	2 files	https://www.tcs.com/investor-relations/...
LTIM.NS	dynamic	4 files	https://www.ltm.com/investors/...
CAP.PA	none	0 files	https://investors.capgemini.com/...

wrote lib/data/ir-discovered.ts: 22 of 23 indexes, 346 files

`plain` means a normal request was enough. `dynamic` means it needed the browser. `none` means that site gave up nothing, which happens and is not an error.

figures, only addresses.

B3. Read the documents

```
npm run harvest:ir
```

spreadsheet reader for `.xlsx`, the PDF reader for `.pdf`, the sentence reader for prose. Maps what it finds onto the 44 concepts, converts to US dollars at a live rate, and refuses anything whose currency or scale the document did not state.

TCS.NS	17 measures	2 files	USD	[revenue depreciation operatingIncome ...]
TECHM.NS	18 measures	8 files	INR at 95.27	[revenue costOfRevenue ...]
CAP.PA	no metric rows could be read			

wrote lib/data/ir-ledger-snapshot.ts: 11 companies, 75 measures, 69 kB

`no metric rows could be read` is a normal outcome for a company that only publishes scanned PDFs. It is not a crash.

B4. Re-read the tracker workbooks

```
npm run ingest:sheets
```

Pulls the two published Google Sheets (the quarterly tracker and the peer universe) and rewrites `lib/data/sector-tracker.ts` and `lib/data/peer-universe.ts`. Takes under a minute.

B5. Train the model

```
npm run train:intent
```

1. Generates 6,084 labelled questions from 116 pattern families, filling each pattern with real company names and real metric names from the universe.
2. Holds out 34 entire families, so the test set contains phrasings the model has never seen rather than rewordings of what it trained on.
3. Extracts features: word unigrams and bigrams, character n-grams, the first word, the first two words, and structural counts. 2,561 after pruning.
4. Trains a multinomial logistic regression for 60 epochs at learning rate 2.0 with L2 regularisation of 2e-5.
5. Scores the model, scores the keyword rules on the same held-out set, then sweeps the confidence floor from 0 to 0.95 to find where the two together do best.
6. Writes the weights.

```
corpus          6084 examples from 116 families
held out        34 families, 1794 examples
features        2561
train accuracy  100.0%
HELD OUT        81.2%   (regex baseline 82.1%)
```

per class on held out:

metric	72.7%	rules	90.2%	n=286
compare	100.0%	rules	74.5%	n=208
rank	100.0%	rules	99.0%	n=208
trend	75.0%	rules	80.8%	n=416
explain	68.8%	rules	68.3%	n=416
product	90.0%	rules	90.0%	n=260

combined policy on held out:

```
floor 0.80 82.7%
chosen floor 0.80 at 82.7%
```

nothing. The three numbers that matter are the model alone (81.2%), the rules alone (82.1%) and the two together (82.7%). The rules beating the model overall is expected and fine: the model exists for `compare`, where it scores 100% against the rules' 74.5%, and the confidence floor keeps it out of the way everywhere else.

array. There is no `.pkl`, no `.onnx`, no `.safetensors` and nothing to load at boot. The array is imported like any other module.

per-class line that fell. The usual cause is a new family that overlaps an intent that already existed.

B6. Check and commit

```
npm run typecheck
npm run build
git add lib/data lib/brain
git commit -m "Refresh the published readings and retrain the router"
git push
```

The push is the deploy. Nothing else is needed.

The whole of part B, if you already have Python set up

```
npm run discover:ir && npm run harvest:ir && npm run ingest:sheets && npm run train:intent
npm run typecheck && npm run build
git add lib/data lib/brain && git commit -m "Quarterly refresh" && git push
```

Budget an hour, mostly waiting.

What can go wrong, and what it means

WHAT YOU SEE	WHAT IT MEANS	WHAT TO DO
ModuleNotFoundError: scrapling	the venv did not install	rerun the pip step in B1 and read its output
Executable doesn't exist at ../chromium	the browser was not downloaded	rerun playwright install chromium
rendered failed: TimeoutError on one company	that site was slow or down	ignore it, the other companies still ran
none 0 files for a company	that site exposed nothing this time	ignore it, that company keeps its previous data
no metric rows could be read	its documents are scans, not text	expected for a few companies, not a crash
403 from an investor site	the host refuses connections from data centres	run the harvest from an ordinary network, not the server
held-out accuracy below 70%	a new training family overlaps an existing intent	check the per-class line that fell

The 403 row is worth understanding. Several investor-relations hosts serve their files to an ordinary connection and refuse one from a hosting provider. This is why the harvest is a local task whose output is committed, rather than something the server does. **Run the harvest from a laptop on a normal network, commit the result, and deploy the commit.**

13. Deployment

```
git push → build → serve on 4870 → reverse proxy
```

```
npm ci && npm run build && npm run start
```

Two things that will otherwise waste an afternoon:

- `proxy_read_timeout 300s` on the reverse proxy. A cold company read is slower than nginx's 60-second default, and the app will look broken when it is not.
- `SEC_USER_AGENT` set to a real contact address. The SEC blocks readers that do not identify themselves.

Full nginx, systemd and Docker configurations are in `README.md` section 3.

14. Dependencies

```
"dependencies": {  
  "next": "^16.3.0",  
  "react": "^19.2.8",  
  "react-dom": "^19.2.8"  
}
```

That is the entire runtime dependency list. No chart library, no PDF library, no spreadsheet library, no ORM, no state manager, no database driver. The charts are hand-built SVG because the content security policy admits no external origin.

15. Known limits

Stated here rather than discovered later.

LIMIT	WHY
Thoughtworks carries no revenue	taken private in 2024, reports nothing
Experian carries no quarterly revenue	its Q1 update gives growth rates and no revenue figure
Infosys and Birlasoft carry no call quotes	their transcripts are set in two columns, which the reader interleaves; rather than print a garbled quote they carry the tracker's written note, labelled as not being the call
Revenue per billable employee is not universal	only companies that publish a utilisation rate have it, and it is not estimated for the others
The cache is per process	behind a load balancer each instance warms separately
The quarterly tracker is a fixed reading	a curated record with management commentary, not a live feed, and the screen says so

16. Repository map

app/	
page.tsx	sector signal
dashboard/company/	one company end to end
dashboard/tracker/	the quarterly sector record
compare/	side by side and cohort position
research/	research pack, private document upload
agents/	diligence workstreams
api/	every endpoint
components/	
charts/	hand-built SVG
dashboards/	one component per screen
ui/	panel, stat block, provenance chip, company picker
lib/	
agents/	registry, analysis, orchestrator
ai/provider.ts	the optional wording model
brain/	ledger, company view, sector rows, intent, engine
core/	fetcher with host allowlist, cache, types
data/	the checked-in readings, all generated
feeds/	SEC, FX, prices, news
financials/	statement model and ratios
pdf/extract.ts	the PDF reader
rag/	retrieval for the ask-the-data panel
research/	document reading and the concept ledger
security/	input sanitising
scraper/	Python. Document discovery only.
scripts/	data refresh and model training

17. Security

- **Host allowlist.** `lib/core/fetcher.ts` connects only to hosts on a fixed list. This is what stops a URL inside a downloaded document from turning the server into an open proxy.
- **No user data stored.** No database, no session. Documents uploaded on the research page are parsed inside that one request and are gone when it returns.
- **Input sanitised** at every endpoint before it reaches a query.
- **Automated clients refused.** `proxy.ts` returns 403 to self-declared bots.
- **Content Security Policy** admits no external origin.

- **Untrusted text is marked.** Anything read from a third-party document is labelled in the citation and never treated as an instruction.